



Week 3: Lab Session

Nested While Loops & Complex Conditions

Duration: 60 minutes — Work in pairs!

Name: _____ Partner: _____

Lab Goals

By the end of this lab, you will be able to:

- Write nested while loops to create 2D patterns
- Use logical operators (and, or, not) in loop conditions
- Generate number pyramids and geometric patterns
- Debug nested loop structures using trace tables

Getting Started

1. Open Thonny on your computer
2. Create a new file called `week3_lab.py`
3. Save it in your CS folder
4. Type your name and your partner's name as comments at the top

1 Exercise 1: Let's Start with Nested Loops (8 minutes)

Type this code exactly as shown and run it:

```
1 # Exercise 1: My first nested loop
2 row = 0
3 while row < 3:
4     print(f"Row {row}: ", end="")
5     col = 0
6     while col < 4:
7         print("*", end=" ")
8         col = col + 1
9     print() # New line after each row
10    row = row + 1
```

 **Checkpoint:** Show your teacher the output before moving on!

Now let's trace through the execution. Fill in this trace table to understand how nested loops work:

Step	Loop	row	col	Condition Check	Output
1	Outer	0	—	row < 3? → 0 ; 3 ✓	—
2	—	0	—	—	Row 0:
3	Inner	0	0	col < 4? → 0 ; 4 ✓	*
4	Inner	0	1	col < 4? → _____	*
5	Inner	0	_____	col < 4? → _____	*
6	Inner	0	_____	col < 4? → _____	*
7	Inner	0	_____	col < 4? → 4 ; 4 ×	—
8	Outer	0	—	print() newline, row = 1	(newline)
9	Outer	1	—	row < 3? → _____	—
10	—	1	—	—	Row 1:
11	Inner	1	_____	col < 4? → _____	*
12	Inner	1	_____	col < 4? → _____	*
13	Inner	1	_____	col < 4? → _____	*
14	Inner	1	_____	col < 4? → _____	*
15	Inner	1	_____	col < 4? → _____	—
Continue the pattern for row = 2...					

Questions about the trace:

- How many times does the inner loop run for EACH outer loop iteration? _____
- What is the total number of stars printed? _____
- When does col reset to 0? _____

Now modify the code:

- Change the outer loop to run 4 times instead of 3
- Change the inner loop to print 6 stars instead of 4
- Replace the "*" with "#"
- What shape did you create? _____

💡 Tip: The outer loop controls rows (vertical), the inner loop controls columns (horizontal)!

2 Exercise 2: Complete the Pattern Program (7 minutes)

Fill in the blanks to make this program create a square pattern:

```

1 # Program to create a 5x5 square of numbers
2 size = 5
3 row = _____ # Initialize row counter
4 while row < _____: # Outer loop condition
5     col = 0
6     while _____ < size: # Inner loop condition

```

```

7         print(row, end=" ")
8         col = ----- # Update inner counter
9         ----- # Don't forget the new line!
10        row = row + 1

```

Test values:

- Your output should show 5 rows
- Each row should have 5 numbers
- All numbers in row 0 should be 0, in row 1 should be 1, etc.

Expected output:

```

0 0 0 0 0
1 1 1 1 1
2 2 2 2 2
3 3 3 3 3
4 4 4 4 4

```

✔ **Checkpoint:** Square pattern working correctly? Excellent!

3 Exercise 3: Experiment with Complex Conditions (10 minutes)

👥 **Pair Programming:** Switch who types every 5 minutes!

Let's explore how logical operators work in loop conditions:

Step 1: Start with this code:

```

1 # Complex condition with AND
2 x = 0
3 y = 0
4 while x < 3 and y < 5:
5     print(f"x={x}, y={y}")
6     x = x + 1
7     y = y + 2
8 print("Loop ended!")

```

Step 2: Try these experiments (run after each change):

1. How many times does the loop run?
Result: _____
2. Change **and** to **or**. How many times does it run now?
Result: _____
3. With **or**, which condition causes the loop to finally stop?
Answer: _____
4. Can you modify the increments so the loop runs exactly 4 times with **and**?
Your changes: _____

💡 **Tip:** With **and**, the loop stops when ANY condition becomes false. With **or**, it stops when ALL conditions become false!

4 Exercise 4: Building a Triangle Pattern (10 minutes)

 **Pair Programming:** Switch who types every 5 minutes!

Let's create a number triangle that grows with each row:

Step 1: Start with this basic structure:

```

1  # Growing triangle pattern
2  row = 1
3  while row <= 5:
4      col = 1
5      # Inner loop should depend on outer!
6      while col <= row:
7          print(col, end=" ")
8          col = col + 1
9      print()
10     row = row + 1

```

Step 2: Experiment with variations:

1. Run the code. What pattern do you see?
Draw it here: _____
2. Change line 6 to print `row` instead of `col`. What changes?
New pattern: _____
3. Change line 5 to `while col <= (6 - row):` What happens?
Pattern type: _____
4. Create a triangle of stars (*) that starts with 5 and decreases to 1:
Hint: Modify the condition in line 5

 **Checkpoint:** Created at least 3 different triangle patterns? Great job!

5 Exercise 5: What Will Happen? (5 minutes)

Before running this code, predict what it will output:

```

1  outer = 0
2  while outer < 2:
3      inner = 0
4      while inner < 3:
5          if outer == 0 or inner == 0:
6              print("*", end=" ")
7          else:
8              print("o", end=" ")
9          inner = inner + 1
10     print()
11     outer = outer + 1

```

Your prediction (draw the pattern):

Now run it.

Actual output: _____

Were you correct? ☐ Yes ☐ No

If not, can you explain why? _____

6 Exercise 6: Mini-Challenge - Multiplication Table (15 minutes)

Create a program that generates a multiplication table!

Your program should:

1. Ask the user for a number between 1 and 10
2. Create a multiplication table for that number
3. Show the table from 1 to 10
4. Format the output nicely

Example Output:

```
Enter a number (1-10): 7
Multiplication table for 7:
-----
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

Starter Help:

```
1  # Multiplication table generator
2  print("Multiplication Table Generator")
3  number = int(input("Enter a number (1-10): "))
4
5  # Validate input with complex condition
6  while number < 1 or number > 10:
7      print("Please enter a number between 1 and 10!")
8      number = int(input("Try again: "))
9
10 print(f"Multiplication table for {number}:")
11 print("-" * 26)
12
13 # Now create the table using a while loop
14 counter = -----
15 while -----:
16     result = -----
17     print(f"{number} x {counter} = {result}")
18     ----- # Don't forget to update counter!
```

 **Pair Programming:** Switch who types every 5 minutes!

Bonus Challenges:

- Can you create a full multiplication grid (like 5x5) using nested loops?
- Add color coding: print even results in one format, odd in another
- Make it print two tables side by side (e.g., for 3 and 4)

 **Checkpoint:** Basic multiplication table working? Show your teacher! Bonus done? Even better!

7 Lab Summary

 **Checkpoint:** Final checkpoint - make sure you have:

- ☐ Completed all exercises with your partner
- ☐ Created at least 3 different patterns
- ☐ Used both **and** and **or** in conditions
- ☐ Built a working multiplication table
- ☐ Saved your file as `week3_lab.py`
- ☐ Shown checkpoints to your teacher

Reflection (2 minutes)

Rate your understanding of today's concepts:

Concept	 Need Help	 Getting It	 Got It!
Nested while loops	☐	☐	☐
Complex conditions (and/or)	☐	☐	☐
Pattern generation	☐	☐	☐
Inner/outer loop coordination	☐	☐	☐

The coolest pattern I made today was: _____

One thing about nested loops I want to practice more:

 **Excellent work with nested loops!**